

Introduction to Queues in C

- A queue in C is basically a **linear data structure** used to store and manipulate data elements.
- It follows the principle of **First In First Out (FIFO)**.

👉 This means:

- The **first element inserted** is the **first one to be removed**.

Characteristics of a Queue

- A queue can handle **multiple data elements**.
- We can access **both ends (front and rear)**.
- Queues are **fast and flexible** for operations.

Types of Queue

1. **Simple Queue**
2. **Circular Queue**
3. **Priority Queue**
4. **Deque (Double Ended Queue)**

Simple Queue

- A simple queue performs basic operations where:
 - **Insertion (enqueue)** occurs at the **rear**
 - **Deletion (dequeue)** occurs at the **front**

👉 Example:

Front → [1][2][3][4] ← Rear

Circular Queue

- In a circular queue, nodes are connected in a **circular manner**.
- The **last node connects back to the first node**.
- It is also called a **Ring Buffer**.
- It is an **abstract data type**.

- It allows:
 - Insertion at the **end**
 - Deletion from the **beginning**

👉 Advantage: Avoids **memory wastage** in simple queues.

Priority Queue

- A priority queue stores elements based on **priority values**.
- Each element has a **preset priority**.

👉 Operations:

- **Insertion** → based on arrival
- **Deletion** → based on **priority (highest priority first)**

Deque (Double Ended Queue)

- In a deque, insertion and deletion can occur at **both ends**:
 - Front
 - Rear

👉 Types of deque:

- Input restricted deque
- Output restricted deque

Operations on Queue

1. **Enqueue()**
 - Adds (stores) an element into the queue
2. **Dequeue()**
 - Removes (accesses) an element from the queue
3. **Peek()**
 - Returns the front element without removing it
4. **isFull()**
 - Checks if the queue is full
5. **isEmpty()**
 - Checks if the queue is empty

Enqueue Operation (Insertion in Queue)

👉 Definition:

Enqueue is the process of **adding an element to the rear of the queue**.

Steps:

1. **Step 1:** Check if the queue is full.
2. **Step 2:** If the queue is full, produce an **overflow error** and exit.
3. **Step 3:** If the queue is not full, **increment the rear pointer** to the next empty space.
4. **Step 4:** Insert the data element at the position where the **rear pointer is pointing**.
5. **Step 5:** Return success.

Dequeue Operation (Deletion in Queue)

👉 Definition:

Dequeue is the process of **removing an element from the front of the queue**.

Steps:

1. **Step 1:** Check if the queue is empty.
2. **Step 2:** If the queue is empty, produce an **underflow error** and exit.
3. **Step 3:** If the queue is not empty, access the element where the **front pointer is pointing**.
4. **Step 4:** **Increment the front pointer** to the next element.
5. **Step 5:** Return success.

Queue Implementation in C

Queue can be implemented in C using:

1. **Array Based**
2. **Linked List Based**

1. Array Based Queue Implementation in C

```
C/C++
```

```
#include <stdio.h>
#define MAX 5
```

```
int queue[MAX];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1)
        front = 0;
    rear++;
    queue[rear] = value;
    printf("%d inserted into queue\n", value);
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("%d deleted from queue\n", queue[front]);
    front++;
}

void peek() {
    if (front == -1 || front > rear) {
        printf("Queue is Empty\n");
        return;
    }
    printf("Front element is %d\n", queue[front]);
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is Empty\n");
    }
}
```

```

        return;
    }
    printf("Queue elements are: ");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();

    dequeue();
    display();

    peek();

    return 0;
}

```

2. Linked List Based Queue Implementation in C

```

C/C++
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node *front = NULL, *rear = NULL;

```

```

void enqueue(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    printf("%d inserted into queue\n", value);
}

void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    struct Node* temp = front;
    printf("%d deleted from queue\n", front->data);
    front = front->next;

    if (front == NULL)
        rear = NULL;

    free(temp);
}

void peek() {

```

```

    if (front == NULL) {
        printf("Queue is Empty\n");
        return;
    }
    printf("Front element is %d\n", front->data);
}

void display() {
    if (front == NULL) {
        printf("Queue is Empty\n");
        return;
    }
    struct Node* temp = front;
    printf("Queue elements are: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    enqueue(100);
    enqueue(200);
    enqueue(300);
    display();

    dequeue();
    display();

    peek();

    return 0;
}

```

Difference Between Array Based and Linked List Based Queue

Array Based Queue	Linked List Based Queue
Uses fixed size array	Uses dynamic memory
Easy to implement	More flexible
May cause overflow if full	No fixed size limit
Wastes memory sometimes	Memory used as needed